PYTHON LIST DATATYPE

Complete Student Notes

Beginner to Intermediate • Concepts • Syntax • Examples • Output • Practice

What you will learn

- List fundamentals and characteristics
- Creating lists, indexing, negative indexing, updating, and slicing
- Adding, removing, and inspecting elements
- Built-in functions, loops, concatenation, and membership
- Unpacking, copying, aliasing, nested lists, and sorting
- List comprehensions, enumerate(), zip(), and common mistakes
- Practice programs and a quick revision checklist

Prepared as classroom-friendly reference notes. Run the examples in Python 3 and predict the output before executing them.

Contents

- 1. What is a list?
- 2. Creating lists
- 3. Indexing and negative indexing
- 4. Updating list elements
- 5. Adding elements: append(), insert(), extend()
- 6. Removing elements
- 7. Common list methods
- 8. Built-in functions
- 9. Traversing lists with loops
- 10. List slicing
- 11. Concatenation, repetition, and membership
- 12. List unpacking
- 13. Aliasing and copying
- 14. Nested lists
- 15. sort() vs sorted()
- 16. List comprehension
- 17. enumerate() and zip()
- 18. Common mistakes and safer patterns
- 19. Practice programs
- 20. Quick revision checklist

1. What Is a List?

A list is a built-in Python collection datatype that stores multiple items in one variable. Lists are written using square brackets `[]`, and items are separated by commas.

- Ordered: elements preserve their position.
- Mutable: elements can be added, removed, or changed.
- Allows duplicates: the same value may appear more than once.
- Mixed types: a list can contain strings, numbers, booleans, and other objects.
- Indexed: each element can be accessed by position, starting at index 0.

Real-life example: student names

```
students = ["Rahul", "Priya", "Amit", "Sneha", "Rohit"]
print(students)
```

Output: `['Rahul', 'Priya', 'Amit', 'Sneha', 'Rohit']`

2. Creating Lists

List of integers

```
numbers = [10, 20, 30, 40, 50]
print(numbers)
```

List of strings

```
names = ["Amit", "Priya", "Rahul"]
print(names)
```

Mixed datatypes

```
data = [101, "Shubham", 85.5, True]
print(data)
```

Empty list and type

```
values = []
print(values)
print(type(values))
```

Output for the final example: `[]` and `<class 'list'>`.

3. Indexing and Negative Indexing

Positive indexing starts at 0 from the beginning. Negative indexing starts at -1 from the end.

```
numbers = [10, 20, 30, 40, 50]
print(numbers[0])   # 10
print(numbers[2])   # 30
print(numbers[4])   # 50
print(numbers[-1])  # 50
print(numbers[-2])  # 40
print(numbers[-5])  # 10
```

Remember: accessing an index outside the valid range raises an `IndexError`.

4. Updating List Elements

```
numbers = [10, 20, 30, 40]
numbers[1] = 200
print(numbers)
```

Output: `[10, 200, 30, 40]`. Assignment replaces the existing element; it does not insert a new element.

5. Adding Elements

append(): add one object at the end

```
nums = [10, 20, 30]
nums.append(40)
print(nums)  # [10, 20, 30, 40]
```

insert(): insert at an index

```
nums = [10, 20, 30]
nums.insert(1, 15)
print(nums)  # [10, 15, 20, 30]
```

extend(): add elements from an iterable

```
nums = [10, 20]
nums.extend([30, 40])
print(nums)  # [10, 20, 30, 40]
```

append() vs extend()

```
a = [1, 2]
a.append([3, 4])
print(a)  # [1, 2, [3, 4]]

b = [1, 2]
b.extend([3, 4])
print(b)  # [1, 2, 3, 4]
```

append() adds its argument as one element. extend() adds each item from the supplied iterable.

6. Removing Elements

Method	What it does
remove(value)	Removes the first matching value; ValueError if not found.
pop()	Removes and returns the last element.
pop(index)	Removes and returns the element at the given index.
clear()	Removes all elements from the list.

```
numbers = [10, 20, 30, 20, 40]
numbers.remove(20)
print(numbers)  # [10, 30, 20, 40]

removed = numbers.pop()
print(removed)  # 40
print(numbers)  # [10, 30, 20]

numbers.clear()
print(numbers)  # []
```

Note: pop() on an empty list raises IndexError. remove() deletes the first matching value, not every occurrence.

7. Common List Methods

Method	Purpose
append(x)	Add x at the end
extend(iterable)	Add each item from an iterable
insert(i, x)	Insert x at index i
remove(x)	Remove first occurrence of x
pop([i])	Remove and return item at i; last item if i is omitted
clear()	Remove all items
index(x)	Return index of first occurrence of x
count(x)	Count occurrences of x
sort()	Sort the list in place
reverse()	Reverse the list in place
copy()	Create a shallow copy

```
numbers = [40, 10, 30, 10, 20]
print(numbers.count(10))  # 2
print(numbers.index(30))  # 2

numbers.sort()
print(numbers)            # [10, 10, 20, 30, 40]

numbers.reverse()
print(numbers)            # [40, 30, 20, 10, 10]
```

Important: sort() and reverse() change the list in place and return None. index() raises ValueError if the value is absent.

8. Built-in Functions with Lists

These are built-in functions, not list methods.

```
marks = [78, 85, 92, 67, 88]
print(len(marks))  # 5
print(min(marks))  # 67
print(max(marks))  # 92
print(sum(marks))  # 410

total = sum(marks)
average = total / len(marks)
print("Total:", total)
print("Average:", average)
```

min(), max(), and sum() require values that can be compared or added appropriately. len() returns the number of top-level elements.

9. Traversing a List with Loops

Using for

```
numbers = [10, 20, 30, 40, 50]
for num in numbers:
    print(num)
```

Using while and an index

```
numbers = [10, 20, 30, 40, 50]
i = 0
while i < len(numbers):
    print(numbers[i])
    i += 1
```

A for loop iterates directly over elements. A while loop can use an index, but the index must be updated to avoid an infinite loop.

10. List Slicing

Syntax: `list_name[start:stop:step]`. The start index is included, the stop index is excluded, and step controls the direction and interval.

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4])  # [20, 30, 40]
print(numbers[:3])   # [10, 20, 30]
print(numbers[3:])   # [40, 50, 60]
print(numbers[::-1]) # [60, 50, 40, 30, 20, 10]
```

A negative step traverses backward. If start/stop positions do not align with the step direction, the result can be an empty list.

11. Concatenation, Repetition, and Membership

```
a = [10, 20, 30]
b = [40, 50]
print(a + b)  # [10, 20, 30, 40, 50]
print(a * 2)  # [10, 20, 30, 10, 20, 30]

names = ["Amit", "Rahul", "Priya"]
print("Rahul" in names)       # True
print("Sneha" not in names)   # True
```

The + and * operators create new lists. Membership checks whether an element is present at the top level of the list.

12. List Unpacking

```
student = ["Rahul", 22, 85]
name, age, marks = student
print(name)   # Rahul
print(age)    # 22
print(marks)  # 85

numbers = [10, 20, 30, 40, 50]
first, *middle, last = numbers
print(first)   # 10
print(middle)  # [20, 30, 40]
print(last)    # 50
```

Without a starred target, the number of variables must match the number of items. The starred target collects remaining items into a list.

13. Aliasing and Copying

Aliasing: both names refer to the same list

```
a = [10, 20, 30]
b = a
b[0] = 100
print(a)  # [100, 20, 30]
print(b)  # [100, 20, 30]
```

Shallow copy: separate outer lists

```
a = [10, 20, 30]
b = a.copy()
b[0] = 100
print(a)  # [10, 20, 30]
print(b)  # [100, 20, 30]
```

A shallow copy does not recursively copy nested mutable objects. For a nested list, the inner lists may still be shared between the original and the copy.

14. Nested Lists

```
marks = [
    [80, 85, 90],
    [70, 75, 88],
    [92, 89, 95]
]
print(marks[0])     # [80, 85, 90]
print(marks[0][1]) # 85
print(marks[2][2]) # 95
```

Think of marks[row][column] as selecting a row first, then an element within that row.

15. sort() vs sorted()

Feature	list.sort()	sorted(list)
Changes original list	Yes	No
Returns	None	New sorted list
Descending order	reverse=True	reverse=True

```
numbers = [40, 10, 30, 20]
result = sorted(numbers)
print(result)   # [10, 20, 30, 40]
print(numbers)  # [40, 10, 30, 20]

numbers.sort(reverse=True)
print(numbers) # [40, 30, 20, 10]
```

16. List Comprehension

A list comprehension creates a new list from an iterable, optionally applying an expression and a condition.

Traditional loop

```
squares = []
for num in range(1, 6):
    squares.append(num * num)
print(squares)  # [1, 4, 9, 16, 25]
```

Equivalent list comprehension

```
squares = [num * num for num in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]

even_numbers = [num for num in range(1, 11) if num % 2 == 0]
print(even_numbers)  # [2, 4, 6, 8, 10]
```

General form: `[expression for item in iterable if condition]`. The if condition is optional.

17. enumerate() and zip()

enumerate(): index and value

```
names = ["Amit", "Priya", "Rahul"]
for index, name in enumerate(names):
    print(index, name)

for index, name in enumerate(names, start=1):
    print(index, name)
```

zip(): corresponding elements from multiple iterables

```
names = ["Amit", "Priya", "Rahul"]
marks = [80, 90, 75]
for name, mark in zip(names, marks):
    print(name, mark)
```

By default, enumerate() starts at 0. zip() pairs items by position and stops when the shortest iterable is exhausted.

18. Common Mistakes and Safer Patterns

Mistake 1: expecting append() to return the updated list

```
numbers = [10, 20]
result = numbers.append(30)
print(result)   # None
print(numbers)  # [10, 20, 30]
```

Mistake 2: removing items while iterating over the same list

```
numbers = [1, 2, 2, 3, 2]
numbers = [num for num in numbers if num != 2]
print(numbers)  # [1, 3]
```

Building a new filtered list avoids index-shifting problems that can occur when removing items during iteration.

Mistake 3: repeated inner lists

```
matrix = [[0] * 3] * 2
matrix[0][0] = 99
print(matrix)  # [[99, 0, 0], [99, 0, 0]]

matrix = [[0] * 3 for _ in range(2)]
matrix[0][0] = 99
print(matrix)  # [[99, 0, 0], [0, 0, 0]]
```

The first expression repeats references to one inner list. The comprehension creates a separate inner list for each row.

19. Practice Programs

- 1. Print every element of a list.
- 2. Find the sum of all elements without using sum().
- 3. Find the largest number without using max().
- 4. Count even and odd numbers.
- 5. Search for a number in a list without using in.
- 6. Count how many times a value appears.
- 7. Create a new list containing only positive numbers.
- 8. Reverse a list without using reverse() or slicing.
- 9. Find duplicate values in a list.

- 10. Remove duplicates while preserving order.
- 11. Find the second-largest distinct number.
- 12. Separate even and odd numbers into two lists.
- 13. Merge two lists without using extend().
- 14. Calculate the average of student marks.
- 15. Move all zeros to the end of a list.

Practice rule: first solve using basic loops and conditions. Then, where appropriate, try a shorter solution using built-in functions or list comprehension.

20. Quick Revision Checklist

- ☐ Can I create empty, numeric, string, and mixed lists?
- ☐ Can I use positive and negative indexing safely?
- ☐ Can I update elements and extract slices?
- ☐ Can I explain append() vs extend() vs insert()?
- ☐ Can I distinguish remove() from pop()?
- ☐ Can I explain list methods versus built-in functions?
- ☐ Can I traverse a list using for and while?
- ☐ Can I explain concatenation, repetition, and membership?
- ☐ Can I unpack a list into variables?
- ☐ Can I distinguish aliasing from shallow copying?
- ☐ Can I access values in nested lists?
- ☐ Can I explain sort() vs sorted()?
- ☐ Can I write a list comprehension?
- ☐ Can I use enumerate() and zip()?
- ☐ Can I identify common list bugs and write safer code?